Designing a Simple OS Task Manager

Computer Systems and Their Fundamentals - Introduction to Operating Systems

Student	Taher Amine ELHOUARI
Checkpoint	Operating Systems - Scheduling, Synchronization, Memory & Disk
Format	Written analysis with calculations, Gantt charts and simulation tables

This submission simulates four core operating-system responsibilities: CPU scheduling, safe shared-resource access, page replacement, and disk scheduling.

Part 1 - Process Scheduling

Processes: P1 arrives at t=0 with burst 5; P2 arrives at t=1 with burst 3; P3 arrives at t=2 with burst 1.

1. FCFS (First-Come, First-Served)

FCFS Gantt Chart

Process	Arrival	Burst	Start	Completion	Waiting
P1	0	5	0	5	0 - 0 = 0
P2	1	3	5	8	5 - 1 = 4
P3	2	1	8	9	8 - 2 = 6

Average waiting time (FCFS) = (0 + 4 + 6) / 3 = **3.33 time units**.

2. Round Robin (quantum = 2)

Round Robin Gantt Chart (q = 2)

Process	Arrival	Burst	Completion	Turnaround	Waiting
P1	0	5	9	9 - 0 = 9	9 - 5 = 4
P2	1	3	8	8 - 1 = 7	7 - 3 = 4
P3	2	1	5	5 - 2 = 3	3 - 1 = 2

Average waiting time (RR) = (4 + 4 + 2) / 3 = **3.33 time units**.

Responsiveness: Round Robin is more responsive even though the average waiting time happens to be equal in this example. P2 first receives CPU time at t=2 and P3 at t=4, whereas under FCFS they first run at t=5 and t=8. RR therefore reduces response time and prevents a long process from monopolizing the CPU.

Part 2 - Process Synchronization

Two processes, P1 and P2, share a variable **counter**. Each process increments the counter 100 times. If counter starts at 0, the expected final value is 200.

What can go wrong without synchronization?

The increment operation is not atomic. It normally consists of reading the current value, adding one, and writing the result back. If P1 and P2 interleave these steps, both may read the same old value and then both write the same incremented value. One update is lost. This is a **race condition**, and the final counter can be less than 200.

Synchronization mechanism: Mutex

A mutex provides mutual exclusion: only one process can execute the critical section that reads and updates the shared counter at a time.

```
shared counter = 0
shared mutex

PROCESS incrementCounter():
repeat 100 times:
lock(mutex)
counter = counter + 1
unlock(mutex)

run P1 and P2 concurrently
wait until both finish
print counter // expected: 200
```

The lock must cover the complete read-modify-write operation. If the lock were acquired only around part of the increment, the race could still occur.

Part 3 - Memory Management

Reference string: **1, 2, 3, 2, 4, 1, 5**. Number of page frames: **3**.

FIFO Page Replacement

Ref	Frame 1	Frame 2	Frame 3	Result
1	1	-	-	Fault
2	1	2	-	Fault
3	1	2	3	Fault
2	1	2	3	Hit
4	4	2	3	Fault (replace 1)
1	4	1	3	Fault (replace 2)
5	4	1	5	Fault (replace 3)

FIFO page faults = 6.

LRU Page Replacement

Ref	Frame 1	Frame 2	Frame 3	Result
1	1	-	-	Fault
2	1	2	-	Fault
3	1	2	3	Fault
2	1	2	3	Hit
4	4	2	3	Fault (replace LRU 1)
1	4	2	1	Fault (replace LRU 3)
5	4	5	1	Fault (replace LRU 2)

LRU page faults = 6.

Comparison: In this specific reference string, FIFO and LRU produce the same number of page faults (6), so neither performs better here. LRU often performs better on workloads with temporal locality because it keeps recently used pages, but this sequence does not give it an advantage.

Part 4 - Disk Scheduling

Initial head position: **53**. Pending requests: **98, 183, 37, 122, 14, 124, 65, 67**.

FCFS Disk Scheduling

Movement	Distance
53 -> 98	45
98 -> 183	85
183 -> 37	146
37 -> 122	85
122 -> 14	108
14 -> 124	110
124 -> 65	59
65 -> 67	2

Total FCFS head movement = 45 + 85 + 146 + 85 + 108 + 110 + 59 + 2 = **640 tracks**.

SSTF (Shortest Seek Time First)

Step	Selected request	Movement
Start	53	-
1	65	12
2	67	2
3	37	30
4	14	23
5	98	84
6	122	24
7	124	2
8	183	59

Total SSTF head movement = 12 + 2 + 30 + 23 + 84 + 24 + 2 + 59 = **236 tracks**.

Conclusion: SSTF is substantially more efficient for this request set: 236 tracks versus 640 for FCFS, a reduction of 404 tracks. SSTF selects the request nearest to the current head position, reducing unnecessary movement. Its trade-off is that requests far from the current region can potentially wait longer (starvation) under heavy workloads.

Final Summary

Area	Result
CPU Scheduling	FCFS avg waiting = 3.33; RR avg waiting = 3.33; RR is more responsive.
Synchronization	Mutex prevents lost updates and ensures counter reaches 200.
Memory	FIFO faults = 6; LRU faults = 6; tie for this reference string.
Disk	FCFS movement = 640 tracks; SSTF = 236 tracks; SSTF is more efficient.

Prepared for the Operating Systems checkpoint. All calculations are shown explicitly so the scheduling and replacement decisions can be verified.